2026年10月19日

OshiruZine-AI

人工知能・機械学習・LLMを中心に、
技術トレンドから実用事例まで幅広く扱う技術系雑誌。

OshiruZine-AI

2026年10月19日 号

目次

1. Claude Code/Codexでのソフトウェア開発の最新ベストプラクティス　*スタンダード（約2000字）*
2. マルチモーダルエージェントの現在地——視覚・音声・操作を横断するAIの統合アーキテクチャ　*特集（約4000字）*
3. EU AI Act全面施行から半年——日本企業が今やるべきコンプライアンス対応チェックリスト　*スタンダード（約2000字）*
4. 推論コスト1/10時代のモデル選定術——蒸留・量子化・ルーティングの最新プラクティス　*ショート（500〜1000字）*

Claude Code/Codexでのソフトウェア開発の最新ベストプラクティス

「AIにコードを書かせる」時代は終わった。2026年の今、私たちは「AIとともにソフトウェアを設計し、構築し、運用する」時代に突入している。その最前線に立つのが、AnthropicのClaude CodeとOpenAIのCodexだ。両ツールはこの1年で劇的に進化し、単なるコード補完ツールから、アーキテクチャ設計・テスト生成・リファクタリング・デプロイメントまでを一貫して支援するAIエージェントへと変貌を遂げた。本コラムでは、2026年10月時点での最新ベストプラクティスを、実務の観点から整理する。

Claude CodeとCodex——2026年の現在地

Claude Codeは2025年の登場以降、急速にターミナルネイティブなAI開発エージェントとしての地位を確立した。2026年に入ってからはClaude 4ファミリーの推論能力を基盤に、大規模コードベースの文脈理解が飛躍的に向上している。プロジェクト全体を「読み解いた」うえでの提案精度は、もはや人間のシニアエンジニアに匹敵するレベルだ。

一方、OpenAIのCodexもCLIエージェントとしてリリースされて以降、GPT-5ベースのコード生成・推論能力と、サンドボックス環境での自律実行を武器に存在感を増している。特にマルチファイル編集やテスト駆動での自律的な修正ループは、実用性において大きな進歩を見せた。

両者に共通するのは、「ただコードを生成する」段階を超え、開発ワークフロー全体をオーケストレーションするエージェントへと進化した点だ。

ベストプラクティス1：CLAUDE.mdとシステムプロンプトの戦略的設計

Claude Codeを使う上で最も効果的かつ見落とされがちなのが、`CLAUDE.md`ファイルの活用だ。これはプロジェクトルートに配置するMarkdownファイルで、Claude Codeがタスク実行時に自動的に読み込む「開発方針書」として機能する。

2026年現在のベストプラクティスとして推奨されるのは、以下の構成だ。

- **アーキテクチャ概要**：システム構成、主要モジュールの責務、データフロー
- **コーディング規約**：命名規則、エラーハンドリング方針、禁止パターン
- **テスト方針**：カバレッジ目標、テストの粒度、モックの使い方
- **セキュリティガイドライン**：入力バリデーション、認証・認可の扱い方
- **やってはいけないこと**：既存APIの破壊的変更禁止、特定ライブラリの使用制限など

Codex側でもシステムプロンプトやリポジトリ内の設定ファイルを通じて同様の制御が可能になっている。重要なのは、AIエージェントに「暗黙知」を明文化して渡すことだ。チームの設計思想をドキュメントとして構造化する行為自体が、人間同士のコミュニケーション品質も向上させる副次効果がある。

ベストプラクティス2：エージェントループの制御と人間の介在ポイント

Claude CodeもCodexも、2026年時点では自律的なエージェントループ——「コード生成→テスト実行→エラー解析→修正→再テスト」のサイクル——を回せるようになっている。しかし、完全に任せきりにするのは危険だ。

実務で効果を上げているチームは、**意図的な介在ポイント**を設計している。

1. **タスク分解段階での承認**：大きな機能実装を依頼する前に、まずAIにタスク分解と実装計画を出力させ、人間がレビューする

2. **アーキテクチャ変更時のゲート**：新しいファイルの作成や依存関係の追加が発生する場合、自動実行を一時停止させる
3. **テスト通過後の意味的レビュー**：テストが通っていても、ビジネスロジックの正しさは人間が判断する

Claude Codeでは権限設定により、ファイル書き込みやコマンド実行を許可制にできる。この粒度を適切に設定することが、安全性と生産性のバランスを取る鍵になる。

ベストプラクティス3：テスト駆動でAIを導く

AIコーディングエージェントの出力品質を最も確実に向上させる手法は、**先にテストを書く（あるいはAIにテストを先に書かせる）**ことだ。これは従来のTDD（テスト駆動開発）の思想と一致するが、AIエージェントとの協業においては特に効果が大きい。

理由は明確で、テストは「期待される振る舞いの形式仕様」として機能するからだ。自然言語での要件記述はどうしても曖昧さが残るが、テストコードは実行可能な仕様として機能する。Claude CodeやCodexはテスト結果をフィードバックとして受け取り、自律的にコードを修正できるため、テストが充実しているほどエージェントの自律ループが正確に回る。

特に効果的なパターンとして、以下のワークフローが挙げられる。

1. 人間が受け入れテスト（E2Eテスト）のシナリオを定義
2. AIにユニットテストとインテグレーションテストを生成させる
3. AIにテストを満たす実装を書かせる
4. テスト通過後、人間がコードレビューと意味的検証を実施

ベストプラクティス4：両ツールの使い分けと併用戦略

2026年10月現在、Claude CodeとCodexはそれぞれ異なる強みを持つ。実践的な使い分け指針を示す。

Claude Codeが優位な領域：大規模既存コードベースの理解と修正、長い文脈を必要とするリファクタリング、ドキュメント生成、複雑なデバッグセッション。ターミナルに常駐してgitやシェルコマンドと統合されている点も、日常的な開発フローとの親和性が高い。

Codexが優位な領域：サンドボックス内での安全な試行錯誤、並列タスク実行、既存のOpenAIエコシステム（API、GPTs）との統合が必要なケース。

先進的なチームでは、探索的な設計フェーズではClaude Codeの対話的な推論力を活用し、明確に定義されたタスクの並列実行にはCodexを投入するという併用戦略を採っている。

まとめ：AIエージェント時代の開発者に求められること

Claude CodeとCodexが成熟した2026年において、開発者に求められるスキルは明確に変化している。「コードを書く速度」ではなく、「AIに正しく意図を伝える能力」「アーキテクチャを判断する能力」「AIの出力を評価する能力」が中核スキルとなった。

CLAUDE.mdの設計、エージェントループへの介在ポイント設計、テスト駆動によるAI制御、ツールの適材適所な使い分け——これらはすべて、AIを「優秀だが文脈を知らない新人エンジニア」として適切にマネジメントする技術だと言える。AIの能力が上がるほど、人間の設計力とレビュー力の重要性はむしろ増していく。その逆説を理解しているチームこそが、2026年の開発生産性競争をリードしている。

マルチモーダルエージェントの現在地——視覚・音声・操作を横断するAIの統合アーキテクチャ

2026年の夏、あるデモ動画がエンジニアコミュニティを騒然とさせた。ユーザーが「この請求書の内容をスプレッドシートにまとめて、不明点があれば取引先に音声で確認して」と話しかけると、AIエージェントが画面上のPDFを読み取り、ブラウザを操作してスプレッドシートに転記し、不足情報を検出すると自ら音声通話を発信して担当者に質問する——すべてが一つのエージェントセッション内で完結していた。

これは概念実証ではない。2026年に入り主要LLMプロバイダが相次いでリリースした**統合マルチモーダルエージェント**の実稼働デモである。画像理解、音声対話、画面操作（Computer Use）という、かつて別々の研究領域だった能力が、単一のエージェントアーキテクチャの中で有機的に結合し始めた。本稿では、この統合がどのような技術スタックの上に成り立っているのかを整理し、開発者が設計時に押さえるべき勘所を示す。

2026年に何が変わったのか——統合の背景

振り返れば、2024年後半から2025年にかけての動きは「モダリティ単体の精度向上」のフェーズだった。OpenAIのGPT-4oがテキスト・画像・音声を統一トークン空間で扱える可能性を示し、GoogleのGeminiシリーズが長大な動画理解を実現し、AnthropicがClaude 3.5で「Computer Use」のβ版を公開した。各プレイヤーはそれぞれの得意領域を伸ばしていたが、エージェントとして複数のモダリティを**同一推論ループ内で自律的に切り替える**仕組みは未成熟だった。

2026年に入って潮目が変わった要因は大きく三つある。

1. **Computer Use系APIの正式GA化と標準化の進展**。 AnthropicのComputer Use APIがGA（一般提供）となり、OpenAIとGoogleもそれぞれ類似のDesktop Action API・Screen Interaction APIを正式リリースした。W3C傘下のコミュニティグループでは、エージェントがWebアプリケーションを操作するための意味的アクション記述フォーマット（Agent Action Protocol: AAP）のドラフト策定が進んでいる。
2. **リアルタイム音声推論の低遅延化**。 音声のストリーミング入力に対して200ミリ秒以下の応答遅延で推論結果を返す「ストリーミングマルチモーダル推論」が実用域に達した。これはモデルアーキテクチャの改善だけでなく、推論サーバーにおけるKVキャッシュの効率的な共有やspeculative decodingの多モーダル対応が寄与している。
3. **統一トークン空間の成熟**。 画像パッチ、音声フレーム、テキストトークン、そしてGUI要素の構造化表現を同一のトークンシーケンスとして処理する「ユニバーサルトークナイザ」のアプローチが複数のプロバイダで採用され、モダリティ間の切り替えコストが劇的に下がった。

これら三つの条件が揃ったことで、「見て、聞いて、操作する」エージェントが現実のプロダクションワークロードとして成立し始めたのである。

統合アーキテクチャの技術スタック全体像

統合マルチモーダルエージェントの技術スタックを、下位レイヤーから順に整理する。

レイヤー1：知覚エンコーダ群（Perception Encoders）

エージェントが外界を「知覚」するための入力処理層である。2026年現在の主流構成は以下の通りだ。

- **視覚エンコーダ：** ViT（Vision Transformer）系の画像エンコーダに加え、スクリーンショットからUI要素のバウンディングボックスとセマンティクスを抽出するUIパーサーが並列に動く。後者はOCRとレイアウト理解を統

合したもので、特にAnthropicのComputer Use v2やGoogleのScreen Interaction APIでは、DOM情報が取得できない場合のフォールバックとして純粋にピクセルベースで操作対象を特定する能力が大幅に向上した。

- **音声エンコーダ：** Whisper系のSTT（Speech-to-Text）を経由する従来パイプラインに代わり、音声特徴量を直接トランスフォーマの埋め込み空間にマッピングするend-to-end方式が主流化。感情やトーン、話者識別といったパラ言語情報も推論に活用される。
- **構造化入力パーサー：** APIレスポンスのJSON、ファイルシステム上のメタデータ、デバイスのセンサーデータなど、テキストでも画像でもない構造化データを統一表現に変換するアダプタ群。

レイヤー2：統一トークン空間と推論コア（Unified Token Space & Reasoning Core）

知覚エンコーダから出力されたマルチモーダル表現は、統一トークンシーケンスとして推論コアに入力される。このコア部分のアーキテクチャには、2026年時点で二つの大きな流派がある。

流派A：Interleaved Attention方式。 テキスト・画像パッチ・音声フレーム・UIセマンティクスのトークンを時系列順にインターリーブ（交互配置）し、単一のトランスフォーマでattentionを計算する。Gemini 2.5系列やGPT-5が採用しているとされるアプローチで、モダリティ横断の推論が自然に行われる利点がある一方、コンテキスト長の消費が大きい。

流派B：Mixture-of-Experts（MoE）ルーティング方式。 各モダリティに特化したエキスパートモジュールを持ち、入力トークンの種類に応じて動的にルーティングする。推論効率に優れ、モダリティごとのファインチューニングが容易という利点がある。Claude 4系列がこの方式を採用していると推測されている。

いずれの方式でも、推論コアが出力するのは「次に取るべきアクション」の構造化表現（ツール呼び出し、発話生成、UI操作コマンド）であり、これがレイヤー3に渡される。

レイヤー3：アクション実行レイヤー（Action Execution Layer）

推論コアの出力を実世界のアクションに変換するレイヤーである。

- **Computer Use実行エンジン：** マウスクリック、キーボード入力、スクロール、ドラッグ&ドロップといったGUI操作を実際に実行する。セキュリティサンドボックス内で動作し、操作結果のスクリーンショットをフィードバックとして知覚エンコーダに返すクローズドループを形成する。
- **音声出力エンジン：** TTS（Text-to-Speech）を経由せず、推論コアから直接音声波形トークンを生成するストリーミング音声出力が主流になりつつある。これにより、応答の自然さと低遅延を両立する。
- **外部ツール・API呼び出し：** Function Calling / Tool Useの枠組みで外部サービスと連携する。2026年にはMCP（Model Context Protocol）の採用が拡大しており、ツール定義の標準化が進んでいる。

レイヤー4：オーケストレーションとメモリ（Orchestration & Memory）

単一の推論ステップではなく、複数ステップにまたがるタスクを管理するレイヤーである。

- **タスクプランナー：** 高レベルの目標を下位タスクに分解し、実行順序を決定する。ReActやPlan-and-Executeパターンをベースに、失敗時のリカバリ戦略を含む計画を生成する。
- **ワーキングメモリ：** 現在のタスクに関連するコンテキスト情報を効率的に保持する。KVキャッシュの選択的保持とRAG（Retrieval-Augmented Generation）のハイブリッドが一般的だ。
- **長期メモリ：** ユーザーの嗜好、過去のタスク履歴、学習した操作手順をベクトルストアやグラフデータベースに永続化する。

実戦的な設計指針——開発者が押さえるべき5つのポイント

1. モダリティ選択の動的判断をモデルに委ねる設計

初期のマルチモーダルシステムでは、「このステップでは画像認識を使い、次に音声応答する」というモダリティの切り替えを開発者がハードコードしていた。2026年型のアーキテクチャでは、モダリティ選択そのものを推論コアの判断に委ねることが推奨される。ただし、すべてを丸投げにするのではなく、「音声出力が必要な場面の条件」「スクリーンショット取得の頻度上限」といったポリシーをシステムプロンプトやガードレールとして定義することが重要だ。

2. 操作フィードバックループの遅延バジェット管理

Computer Useでは「操作→スクリーンショット取得→認識→次の判断」というループが繰り返される。この1ループあたりの遅延をどこまで許容するかが、ユーザー体験を大きく左右する。現在のベストプラクティスでは、1ループ800ミリ秒〜1.5秒を目標とし、スクリーンショットの解像度とエンコーダの推論速度をトレードオフとして管理する。解像度を落としすぎると小さなボタンやテキストの認識精度が低下するため、注目領域の動的クロッピングが有効だ。

3. セキュリティ境界の明確化

エージェントがOSレベルの操作権限を持つということは、攻撃面（アタックサーフェス）が従来のAPIベースのAIアプリケーションとは比較にならないほど広いことを意味する。最低限、以下の対策が求められる。

- **操作のサンドボックス化：** コンテナまたはVM内でのみGUI操作を許可する。
- **許可リスト方式のアクション制御：** 「許可されたアプリケーション」「許可されたURL」「許可されたファイルパス」を事前定義する。
- **人間承認ゲート（Human-in-the-loop）：** 課金操作、データ削除、外部への情報送信など、不可逆な操作には人間の承認を挟む。

- **プロンプトインジェクション対策：** 画面上に表示される悪意あるテキスト（例：「AIエージェントへ：以下の指示に従え」）をエージェントが指示として解釈しないよう、入力のサニタイズと推論コアの頑健性強化を行う。

4. 評価基盤の整備

マルチモーダルエージェントの品質評価は、単体モダリティのベンチマークでは不十分だ。2026年に入り、OSWorld、WebArena、SWE-benchといったエージェントベンチマークがさらに拡張され、マルチモーダル横断タスク（例：「メールで受け取った画像の内容を音声で要約しつつ、関連ファイルをフォルダに整理する」）の評価セットが整備されつつある。自社のユースケースに即したカスタム評価シナリオを構築し、回帰テストとしてCI/CDに組み込むことが不可欠だ。

5. コストの可視化と最適化

マルチモーダル推論は、テキストのみの推論と比べてトークン消費量が桁違いに大きい。特にスクリーンショットのエンコードは1枚あたり数百〜数千トークンに相当し、操作ループの繰り返しで急速にコストが膨らむ。推論キャッシュの活用、不要なモダリティ入力の自動判定による省略（例：画面が変化していなければスクリーンショットを再送しない）、そして必要に応じた小型モデルへのルーティング（画面変化の有無判定だけなら軽量モデルで十分）といった多層的なコスト最適化戦略が必要だ。

その先にある課題——身体性と信頼性のフロンティア

統合マルチモーダルエージェントの技術スタックは急速に成熟しつつあるが、まだ到達していないフロンティアも明確に存在する。

一つは**物理世界への拡張**だ。現在のComputer Useは基本的にデジタル画面上の操作に限定されているが、ロボティクスとの統合により、カメラで現実空間を認識し、ロボットアームや自律移動体を制御する方向への進化が見え始めている。2026年後半には、いくつかの研究グループが「デスクトップ操作と実ロボット制

御を同一エージェントセッション内で切り替える」デモを発表しており、デジタルと物理の境界はさらに曖昧になるだろう。

もう一つは**信頼性と説明責任**だ。エージェントが自律的に多段階の操作を行う場合、途中のどのステップで誤った判断が行われたのかをトレース可能にする仕組みが必須になる。操作ログの構造化、各ステップでの推論根拠（Chain-of-Thought）の保存、そして監査可能なアクション履歴の提供が、エンタープライズ導入においては技術的な精度以上に重視される場面も多い。

おわりに

2026年は、マルチモーダルAIが「マルチモーダルであること自体」を売り文句にする時代の終わりを告げた年として記憶されるかもしれない。画像を理解できる、音声で対話できる、画面を操作できる——それらはもはや個別の機能ではなく、エージェントが実世界のタスクを遂行するための**当然の前提条件**になりつつある。

問われているのは、統合された知覚とアクションの能力を、いかに安全に、効率的に、そして信頼できる形でデプロイするかという設計の技術だ。本稿で整理した技術スタックと設計指針が、読者の皆さんが次のエージェントアーキテクチャを構築する際の地図となれば幸いである。

EU AI Act全面施行から半年――日本企業が今やるべきコンプライアンス対応チェックリスト

2026年8月2日、EU AI Act（人工知能規則）の高リスクAIシステムに関する義務規定が全面適用となった。それから約2か月半――すでに欧州市場向けにAIシステムを提供する複数の日本企業に対し、欧州委員会および各国所管当局から「技術文書の提出要請」が届き始めている。「うちはEUに拠点がないから関係ない」という認識は、もはや通用しない。本コラムでは、域外適用の現実を踏まえ、エンジニアリングの現場で"最低限これだけは"押さえておくべき対応事項をチェックリスト形式で整理する。

域外適用はもう「他人事」ではない

EU AI Actの第2条は、AIシステムの出力がEU域内で使用される場合、提供者（provider）の所在地にかかわらず規則が適用されると明記している。つまり、日本国内で開発・運用しているAIでも、その推論結果がEU域内の自然人に影響を及ぼすなら対象になる。2026年9月にはEU AI Office（AIオフィス）が域外事業者向けのガイダンス第2版を公表し、「EU域内の認定代理人（authorised representative）の選任義務」について具体的な手続きフローが示された。SaaS型AIサービスをグローバル展開している企業は、まず自社が規則の適用対象かどうかを判定するところから始めなければならない。

チェックリスト①：リスク分類の実務判断

EU AI Actは、AIシステムを「禁止（Unacceptable）」「高リスク（High-risk）」「限定リスク（Limited risk）」「最小リスク（Minimal risk）」の4段階に分類する。問題は、自社システムがどこに該当するかの判断が想像以上に難しい点だ。

- **Annex IIIの用途リストとの照合**：採用選考支援、信用スコアリング、法執行、重要インフラ管理など、Annex IIIに列挙された8カテゴリ・具体的用途と自社システムの機能を一つずつ突き合わせる。
- **「安全コンポーネント」該当性の確認**：Annex Iに列挙されたEU整合法令（機械規則、医療機器規則など）の対象製品に組み込まれるAIは、それ自体が高リスクに分類される。組込みAIを提供しているエンジニアは特に注意が必要だ。
- **Art.6(3)の例外規定の活用**：高リスクに分類される用途であっても、意思決定の補助にとどまり人間が最終判断する場合など、一定の条件下で例外的に高リスク分類から除外される可能性がある。ただし、この「例外主張」には文書化された根拠が求められる。2026年9月のガイダンスでは、除外を主張する場合の立証責任は提供者側にあることが改めて強調された。

チェックリスト②：技術文書の整備

高リスクAIシステムの提供者には、Art.11およびAnnex IVに基づく包括的な技術文書の作成・維持義務がある。最低限、以下の項目を文書化しておく必要がある。

- **システムの全体設計と動作原理**：アーキテクチャ図、使用アルゴリズムの種類、学習手法の概要
- **訓練・検証・テスト用データセットの記述**：データの出所、前処理手順、バイアス検出・緩和の方法、データガバナンスの体制
- **性能指標とその測定方法**：精度、堅牢性、サイバーセキュリティ耐性、公平性の各メトリクスと評価プロトコル
- **リスクマネジメントシステム（Art.9）の記録**：特定したリスク、採用した緩和策、残留リスクの評価
- **人間による監視（human oversight）の設計仕様**：Art.14に基づき、運用者がシステムの出力を理解し、介入・停止できる仕組みの技術的詳細

- **ログ記録機能の仕様**：Art.12に基づく自動ログの保持期間、記録項目、改ざん防止措置

エンジニアにとって特に負荷が高いのはデータセット関連の文書化だ。訓練データの来歴（data lineage）を遡及的に整備するのは極めて困難なので、今後の開発パイプラインには最初からデータカードやモデルカードの生成を組み込むことを強く推奨する。

チェックリスト③：適合性評価と市場投入後の監視

高リスクAIシステムの多くは「内部適合性評価」（自己評価）で市場投入が可能だが、一部の生体認証系システムなど、第三者機関（Notified Body）による評価が必須となるケースがある。

- **適合性評価手続きの選定**：Annex VIの内部管理手続き、またはAnnex VIIの第三者評価のどちらが適用されるかを確認する。
- **CEマーキングとEU適合宣言書の準備**：高リスクAIシステムにはCEマーキングの貼付義務がある。EU適合宣言書（Declaration of Conformity）にはArt.47で定められた必須記載事項を漏れなく含める。
- **EU AIデータベースへの登録**：Art.71に基づき、高リスクAIシステムは市場投入前にEU公開データベースへの登録が必要。2026年8月の全面適用開始と同時にデータベースの登録受付が本格稼働しており、登録なしの市場投入は違反となる。
- **市場投入後監視（Post-Market Monitoring）計画の策定**：Art.72に基づき、システムの性能劣化やバイアスのドリフトを継続的に監視する体制を構築する。重大インシデント発生時には当局への報告義務（Art.73）もある。

エンジニアが「今日から」できること

法務・コンプライアンス部門との連携はもちろん重要だが、技術文書の大半はエンジニア自身にしか書けない。まずは以下の3つから着手してほしい。

1. **自社AIシステムの棚卸しとリスク分類マッピングを行う**。スプレッドシート1枚でいい。システム名、用途、提供先地域、Annex III該当性を一覧化するだけで景色が変わる。
2. **MLOpsパイプラインにデータカード・モデルカードの自動生成を組み込む**。後から文書を作るコストは開発時の数倍になる。
3. **ログ記録と監視基盤の設計を見直す**。Art.12が要求するログ要件を既存の可観測性（observability）スタックでカバーできるか、ギャップ分析を実施する。

日本政府も2026年春に「AI事業者ガイドライン」の改訂版を公表し、EU AI Actとの相互運用性を意識した枠組みの整備を進めている。国内法制度が追いつくのを待つのではなく、グローバル標準への対応を先行して進めることが、技術的負債を減らし、競争優位を確保するための最短ルートだ。規制対応は「守り」ではなく、信頼されるAIを作るための「設計品質の向上」だと捉え直すことが、エンジニアにとっての最も健全なスタンスだろう。

推論コスト1/10時代のモデル選定術——蒸留・量子化・ルーティングの最新プラクティス

「GPT-4級の品質を、GPT-3.5以下のコストで」——半年前なら誇大広告だったこのフレーズが、2026年後半の今、エンジニアリングの現実になりつつある。推論コストを桁で削る4つの柱を、プロダクション視点で整理しよう。

蒸留——"教師の知恵"を小型モデルに凝縮する

大規模モデルの出力を教師データとして小型モデルを訓練する蒸留は、もはや定番手法だ。2026年に入り、タスク特化の合成データで蒸留する**On-Policy Distillation**が主流化した。汎用ベンチマークではなく、自社のユースケースに絞った数万件の教師出力で3Bクラスのモデルを仕上げれば、70B超のモデルと遜色ない精度が出るケースが報告されている。コスト比はざっくり1/10〜1/20。最初の一手として検討すべき手法だ。

量子化——INT4からFP4へ、精度と速度の両取り

重みをINT4に落とす量子化は広く普及したが、今年はNVIDIA Blackwell世代の**FP4ネイティブ対応**が転機になった。GPTQ/AWQに加え、FP4で学習時から量子化を意識するQuantization-Aware Training（QAT）の適用事例が増え、INT4比で推論スループットをさらに30〜50%改善する報告が相次ぐ。蒸留済み小型モデル＋FP4量子化の組み合わせは、現時点で最もコスパの高い構成の一つだ。

スペキュラティブデコーディング——"下書き＋検算"で遅延を圧縮

小型のドラフトモデルでトークンを先読みし、大型モデルが一括検証する**スペキュラティブデコーディング**は、レイテンシ削減の切り札として定着した。最近では蒸留モデルをドラフターに流用し、アクセプタンス率を80%以上に引き上げる設計が一般的になっている。体感速度が2〜3倍になるため、チャットやコード補完などストリーミング用途で威力を発揮する。

モデルルーター——リクエストごとに"最適な脳"を選ぶ

そして今、最もホットなのが**モデルルーティング**だ。リクエストの難易度や種別をルーターが判定し、簡単なタスクは3Bモデル、複雑な推論は70B以上のMoEモデルへ振り分ける。Martianの「Model Router」やOpenRouterのコスト最適化APIに加え、OSSではRouteLLMの採用事例が急増中だ。実運用では全リクエ

ストの70〜80%が小型モデルで処理可能なことが多く、加重平均コストを劇的に押し下げられる。

実務での組み合わせ方

これらは排他ではなく**重ね掛け**が基本だ。推奨は以下のステップとなる。

1. **蒸留**でタスク特化の小型モデルを作る
2. **FP4量子化**でさらにスループットを上げる
3. **ルーター**で大型モデルとのハイブリッド構成を組む
4. 大型モデル側に**スペキュラティブデコーディング**を適用しレイテンシを削る

すべてを適用すれば、推論コスト1/10は決して夢物語ではない。重要なのは「精度を落とさずコストだけ落とす」設計思想だ。まずは自社トラフィックの難易度分布を可視化するところから始めてみてほしい。
